CALRec: Contrastive Alignment of Generative LLMs for Sequential Recommendation

Yaoyiran Li*†
University of Cambridge
United Kingdom
yl711@cam.ac.uk

Xiang Zhai*
Google
USA
xiangzhai@google.com

Moustafa Alzantot
Google
USA
mfarid@google.com

Keyi Yu
Google
USA
yukeyi@google.com

Ivan Vulić
University of Cambridge
United Kingdom
iv250@cam.ac.uk

Anna Korhonen
University of Cambridge
United Kingdom
alk23@cam.ac.uk

Mohamed Hammad
Google
USA
mibra@google.com

Abstract

Traditional recommender systems such as matrix factorization methods have primarily focused on learning a shared dense embedding space to represent both items and user preferences. Subsequently, sequence models such as RNN, GRUs, and, recently, Transformers have emerged and excelled in the task of sequential recommendation. This task requires understanding the sequential structure present in users' historical interactions to predict the next item they may like. Building upon the success of Large Language Models (LLMs) in a variety of tasks, researchers have recently explored using LLMs that are pretrained on vast corpora of text for sequential recommendation. To use LLMs for sequential recommendation, both the history of user interactions and the model's prediction of the next item are expressed in text form. We propose CALRec, a two-stage LLM finetuning framework that finetunes a pretrained LLM in a two-tower fashion using a mixture of two contrastive losses and a language modeling loss: the LLM is first finetuned on a data mixture from multiple domains followed by another round of target domain finetuning. Our model significantly outperforms many state-of-the-art baselines (+37% in Recall@1 and +24% in NDCG@10) and our systematic ablation studies reveal that (i) both stages of finetuning are crucial, and, when combined, we achieve improved performance, and (ii) contrastive alignment is effective among the target domains explored in our experiments.

*Both authors contributed equally to this research. Corresponding authors: Yaoyiran Li {yl711@cam.ac.uk} and Xiang Zhai {xiangzhai@google.com}.
†This work has been done during the author's internship at Google.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
RecSys '24, October 14–18, 2024, Bari, Italy
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0505-2/24/10
https://doi.org/10.1145/3640457.3688121

CCS Concepts

• **Information systems** → **Recommender systems**; • **Computing methodologies** → **Neural networks**.

Keywords

Sequential Recommendation, Large Language Models, Contrastive Learning

ACM Reference Format:
Yaoyiran Li, Xiang Zhai, Moustafa Alzantot, Keyi Yu, Ivan Vulić, Anna Korhonen, and Mohamed Hammad. 2024. CALRec: Contrastive Alignment of Generative LLMs for Sequential Recommendation. In *18th ACM Conference on Recommender Systems (RecSys '24), October 14–18, 2024, Bari, Italy*. ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3640457.3688121

1 Introduction and Motivation

Recommender systems aim to understand users and items and use the learned knowledge to recommend relevant items for future user interactions. Traditional collaborative filtering methods [15, 25, 42] map item and user IDs into a common latent embedding space learned from individual user-item interactions, but these embeddings are often static and fail to capture the dynamic nature of user interests. Recent exploration in sequential recommender employs RNN [16], CNN [54] and the transformer [22, 45] to directly learn the time-evolution of user behaviors. The transformer architecture [10, 49] stands out for its efficient capability of modeling sequences through the self-attention mechanism, which learns the correlation or causal relationships between different tokens and their influence on future tokens. In a large body of recommender systems (RecSys) research, each input position (token) corresponds to a single user activity, typically a user's interaction (e.g., purchase) with an item [18, 22, 45, 57, 59]. More specifically, some approaches rely on item IDs, each represented by a unique token, and model a user's interaction history as a sequence of item ID tokens [22, 45]; other efforts combine more sophisticated features, such as the text embedding of item descriptions, to capture richer content information for each item [57, 59].

Motivated by the global paradigm shift towards autoregressive Language Large Models (LLMs) [1, 2, 46, 47] with successful applications in various tasks and domains [3, 11, 39, 55], we focus on leveraging generative LLMs for sequential recommender systems. Pretrained LLMs learn knowledge from large text corpora and are expected to perform reasonably well in general tasks, and their performance on specific tasks further improves after fine-tuning with domain-specific data. Several studies have explored utilizing LLMs in recommender systems [52], such as framing sequential recommendation as sentence-completion [27] or question-answering problems [56].

Despite the impressive language understanding capabilities of pretrained LLMs, they require fine-tuning for highly specific tasks like sequential recommendation. An LLM should be ideally fully fine-tuned [23, 51] to learn to comprehend domain-specific flattened text sequences and the underlying user interest evolution. Similar to most sequence modeling approaches like SASRec [22], BERT4REC [45], Recformer [26], and GPT4Rec [27], we adopt next-item prediction as the sequential recommendation task.

Based on sentence-completion [27] or question-answering [56] styled prompting, our template design is further inspired by the idea of few-shot learning [7]. Specifically, we prefix user sequence and each item in the user sequence with distinctive text indicators that **1)** allow the model to better understand data pattern and text format and **2)** increase the LLM response coherence of the input sentence. We further propose a Contrastive Aligned Generative LLM Recommendation (CALRec) framework to adapt an LLM for sequential recommendation, inspired by the effectiveness of contrastive learning (CL) in mapping-based RecSys [18, 26] and CL research in NLP and other areas [13, 28]. CALRec features the following:

- Pure text input and text output with advanced prompt design.
- A mixed training objective that combines customized next-item generation objective and auxiliary contrastive objectives.
- A two-stage LLM fine-tuning paradigm consisting of a multi-category joint fine-tuning stage followed by a category-specific fine-tuning stage.
- A novel quasi-round-robin BM25 retrieval approach for item retrieval.

Similar to previous work [22, 26, 45], we leverage the Amazon Review dataset [35] and compare our results to open-source state-of-the-art (SotA) baselines. We observed a non-negligible percentage of consecutive identical activities by the same users in the raw datasets which might trivialize the next-item prediction task. We thus train and evaluate the model on deduplicated sequences and propose to adopt 'Last Item Repeater', a training-free, text-statistic baseline for model comparison (§4 and §6). We also revisit the existing evaluation metrics of the previous works, and propose to report both optimistic and pessimistic metric scores for text-based RecSys (§4). Through comprehensive experiments, we discover that our CALRec method demonstrates superior performance and outperforms existing SotA by a considerable margin in most evaluation metrics. A systematic ablation study indicates the effectiveness of the two-stage training paradigm, our contrastive alignment, and our template design among the various data categories examined in our experiments.

Finally, we briefly summarize the contribution of our study:

- We proposed CALRec, a novel sequential recommendation framework that features advanced prompt design, a two-stage training paradigm, a combined training objective, and a quasi-round-robin BM25 retrieval approach.
- We conduct comprehensive experiments, ablation study and further analyses and show that **1)** our approach outperforms existing baselines in most metrics by a considerable margin; **2)** the main components of our approach are effective. We also discuss **3)** the characteristics of our output and our limitations.
- We revisit data preprocessing and evaluation metrics in this field.

2 Related Work

Traditional recommender systems like Collaborative Filtering [15] learn relevance through user-item co-occurrence. Early collaborative filtering approaches like Matrix Factorization (MF) [25] are ID-only where both user and item IDs are embedded in a shared embedding space. Later MF improvements incorporated other attributes [43] such as time and location. Nevertheless, these approaches did not capture the sequential nature of user behavior which gave rise to a class of Sequential Recommenders. The initial Sequential Recommenders, similar to MF, were ID-only models, exemplified by GRU4Rec [16], SASRec [22], and Bert4Rec [45]. Subsequent research expanded these models by integrating additional attributes beyond IDs, leading to the development of methods such as FDSA [57], S3-Rec [59], and UniSRec [18]. Due to the challenges associated with IDs in real world Recommender Systems [43], researchers explored removing them in favor of attribute-only models [18, 26]. However, this shift led to quality issues, as the valuable 'memorization characteristics' provided by IDs were lost. Another thread of research explored deriving IDs from attribute embedding [17, 38, 44], which mitigates some of the challenges associated with random IDs, albeit at the cost of increased system complexity.

With the advent of pretrained LLMs, which include encoder-only models like BERT [10, 26], encoder-decoder models like T5 [37], and decoder-only models like PaLM [2, 9], GPT [7], and LLama [48], researchers explored leveraging these backbones in sequential recommendation tasks, harnessing the extensive 'knowledge' encoded within their parameters. Some approaches utilized zero-shot or few-shot methods [19, 33, 50], while others extract item text embeddings from LLMs, both with frozen LLM parameters.

Previous work also explored fine-tuning encoder-only, encoder-decoder or decoder-only LLMs to incorporate recommendation knowledge into the LLMs [4, 20, 23, 26, 32, 37, 52, 56, 60]. P5 [14] fine-tuned a pre-trained T5 [37] model to solve various common RecSys tasks, including sequential recommendation. In contrast to P5, which frames the sequential recommendation task as an ID generation problem, our approach focuses on generating item attribute text and features a two-tower training framework, enhancing model's understanding of items with their attributes within the context of user-item interaction sequences. GPT4Rec [27] fine-tunes a pre-trained GPT-2 model to generate the next-item titles. Similar to GPT4Rec, we use BM25 [40] to search for items in the

item corpus. However, unlike GPT4Rec which was directly fine-tuned for next-item generation as a sentence completion task on each target domain respectively, we are among the first to explore using the contrastive loss as an auxiliary objective to fine-tune an autoregressive LLM on the sequential recommendation task, and our work also features a multi-stage fine-tuning framework, enabling our model to generalize to multiple target domains. We include the list of baseline models we compare against and their details in §4.

3 Methodology

3.1 Preliminaries and Terminology

We assume a set of users $\mathcal{U}$, where a user $u \in \mathcal{U}$ is associated with an interaction sequence comprising n items $(I_1, I_2, ..., I_n)$ arranged in chronological order (note that the value of n may vary across different users). The *sequential recommendation* task is then defined as follows: given the user's historical interaction sequence $I_{1:n-1} = (I_1, I_2, ..., I_{n-1})$, a sequential recommender aims to predict the target item I_n, the last item in the user's full interaction sequence. In our study, we use pure text to represent each item and user interaction sequence (§3.2), and we adapt autoregressive LLMs pretrained on large corpora of text for sequential recommendation. Specifically, we pose the task as a sequence-to-sequence generation task where the input is the text description of $I_{1:n-1}$ and the expected output is that of I_n. Our primary motivation for formulating the problem entirely within the text domain is to exploit the rich textual information inherent in real-world data and to capitalize on LLMs' strong language understanding and reasoning capabilities.

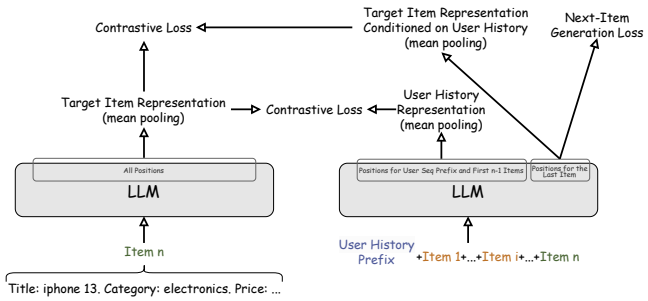

Figure 1: CALRec: Training Framework.

3.2 Data Format and Template Design

In our setup, items and user sequences are provided exclusively in text format. Instead of treating per-item features separately as in traditional recommender models [57, 59] and inspired by Li et al. [26], we flatten each item's semantically meaningful attributes into a single sequence of text tokens, where the attributes are respectively formatted in the style of "`Attribute Name: Attribute Description`" and then are concatenated together. Typical attributes include the title, ratings, keywords, price, and others, exemplified by the title attribute "`Title: iPhone 13 Pro Max`". We discard non-semantic ID attributes such as item hashing IDs or user IDs which may add noise to the LLM input. Missing attribute descriptions are replaced with the wildcard text "`Unknown`". For the user sequence, different from Li et al. [26], we additionally use a user prefix "`This is the summary of a user's purchase history.`" and per-item prefixes "`\nThe first/next item bought is as follows.`" ("`\n`" is not applied on the first item) to separate items. The user sequence prefix signature facilitates LLMs in differentiating between single item input and user sequence input. The per-item prefix signature aids LLMs in segmenting distinct items within a user history sequence, thereby enhancing task comprehension. In addition, we end the user history sequence with the same item prefix, prompting the LLM to predict the next purchase as a sentence completion task. This design is inspired by the idea of 'few-shot in-context learning' [7, 29, 30]:[1] since the same item prefix is also seen in the history sequence, our approach is beneficial for the LLM to generate text that follows the output format. A concrete example of the full prompt is provided as follows[2]:

```
———————— [USER PURCHASE HISTORY SEQUENCE] ————————
This is the summary of a user's purchase history. The first item bought
is as follows. Title: The Hillman Group 591520 Small Wire Nail and
Brad Assortment, 260-Pack. Category: Tools & Home Improvement. Brand:
Hillman. Price: Unknown.
The next item bought is as follows. Title: Pro Flex Super Flexible
Elastomeric Sealant Acrylic Clear Paintable 10 Oz. Category: Tools &
Home Improvement. Brand: Geocel. Price: $11.22.
The next item bought is as follows.
———————— [TARGET ITEM SEQUENCE] ————————
Title: Trimaco Llc RF36 35-Inch by 166-Feet Rosin Flooring Paper, Red.
Category: Tools & Home Improvement. Brand: Trimaco. Price: Unknown.
```

Different from our method, many previous text-based approaches do not specify an item prefix and simply leverage a one-off user history suffix [27, 56] (some approaches even do not specify any user suffix at all [26]).

3.3 Two-Stage Fine-Tuning

Previous work [18, 26] indicates that pretraining a RecSys on a large volume of data (various data categories) provides favorable inductive bias for downstream tasks (a specific and smaller-scale data category for evaluation after further fine-tuning on them). Analogously, based on an LLM already pretrained on large, diverse text corpora, we propose a two-stage fine-tuning framework that consists of Multi-Category Joint Fine-Tuning (Stage I) and Category-Specific Fine-Tuning (Stage II): the latter benefits from the former as a result of transfer learning. Our Stage I fine-tuning allows the model to adapt to the sequential recommendation problem setting and learn data patterns in a category-agnostic fashion. Inspired by Xue et al. [53], we adopt a sampling rate for each data category proportional to $|\mathcal{U}|^{0.3}$, where $|\mathcal{U}|$ is the size (number of users) of each category, for the purpose of combating the data imbalance issue, preventing a few extra-large categories from dominating the Stage I fine-tuning set. After deriving the multi-category fine-tuned model, we further adapt the model to each category respectively via Category-Specific Fine-Tuning (Stage II). In both stages, the training objective is the same, introduced in §3.4.

[1] Our approach does not conform to the exact concept of 'few-shot' learning by definition, since our input data is always from a single user.

[2] During training, the user history and the target item sequences (from the training set) are concatenated. At inference, only the user history sequence (from the validation/test set) serves as the LLM input.

3.4 Training Objectives

Next-Item Generation Objective. The primary objective employed for LLM fine-tuning is the Next-Item (Text) Generation *(NIG)* Objective. NIG aims to generate the text description of the target item given the text description of the previous items in a user's history. Denote the text-tokenized user sequence (for a user) as $\mathrm{T} = (t_1, t_2, ..., t_l)$, where l is the tokenized sequence length. The first m tokens belong to all the items in the user sequence except for the last item (i.e., the target item), followed by $l - m$ tokens for the target item. We propose this next-item generation objective to adapt an LLM for sequential recommendation:

$$\mathcal{L}_{NIG} = \mathop{\mathbb{E}}_{\mathrm{T} \sim p(\mathrm{T})} \left[\sum_{j=m+1}^{l} \log P(t_j | t_{1:j-1}; \theta) \right] \tag{1}$$

where p(T) denotes the data distribution, and θ is the whole set of trainable parameters of the chosen LLM.

Auxiliary Contrastive Alignment. One shortcoming of the NIG is that it deals with the input and output on the token level rather than on the item/user level. Therefore, in addition to the token-level NIG objective, we also investigate auxiliary contrastive objectives that operate on the item/user level. Specifically, we adopt a two-tower training framework where one tower only takes in the target item, and the mean-pooled feature in the final hidden layer is denoted as $\mathbf{v}^T$. The second tower takes the whole user sequence and similarly we denote the mean-pooled feature corresponding to the user history sequence as $\mathbf{v}^U$ and the feature for the target item conditioned on the user history as $\mathbf{v}^{T|U}$ (see also Fig. 1). We experiment with two contrastive losses for user- and item-level alignments following the InfoNCE loss [36], a standard and robust choice in contrastive learning research [8, 13, 28, 31]. Below, we present the calculation of these losses on a batch of data:

$$\mathcal{L}_{TT} = -\frac{1}{N_b} \sum_{i=1}^{N_b} \log \frac{\exp(\cos(\mathbf{v}_i^{T|U}, \mathbf{v}_i^T)/\tau_c)}{\sum_{j=1}^{N_b} \exp(\cos(\mathbf{v}_j^{T|U}, \mathbf{v}_i^T)/\tau_c)}, \tag{2}$$

$$\mathcal{L}_{UT} = -\frac{1}{N_b} \sum_{i=1}^{N_b} \log \frac{\exp(\cos(\mathbf{v}_i^{U}, \mathbf{v}_i^T)/\tau_c)}{\sum_{j=1}^{N_b} \exp(\cos(\mathbf{v}_j^{U}, \mathbf{v}_i^T)/\tau_c)}, \tag{3}$$

where in-batch negative samples are adopted, N_b is the batch size (number of user sequences in a training batch), $\cos(\cdot, \cdot)$ calculates the cosine similarity between two input vectors, and τ_c is the temperature for contrastive alignments.

Our final training objective is a mixture of $\mathcal{L}_{NIG}$ and contrastive losses as shown in Fig. 1:

$$\mathcal{L}_{CALRec} = (1 - \alpha - \beta)\mathcal{L}_{NIG} + \alpha\mathcal{L}_{TT} + \beta\mathcal{L}_{UT}. \tag{4}$$

3.5 Quasi-Round-Robin BM25 Retrieval

At inference, we prompt the model with the text description of user's history of interactions (see also §3.2). To obtain more than one candidate next-item prediction, we conduct temperature sampling [12] to generate N_{gen} pieces of text as N_{gen} predictions for the next item. Each of the N_{gen} predictions is associated with its sequence score s (i.e., log probability, which is a negative real number typically in the range of $(-10, 0)$, as observed on our data).

Algorithm 1: Quasi Round Robin BM25 Retrieval

Input : *samples*, output texts sampled from the LLM (size = N_{gen});
logprobs, LLM sequence scores of *samples* (size = N_{gen});
C, corpus of candidate items (size = N_c);
N_{preds}, a hyperparameter that determines the number of generated text samples used for BM25 matching;
ϵ, a hyperparameter that controls the modulation of BM25 scores with LLM sequence scores.
Output : The sorted list of item predictions.

```
// Sort samples by score and remove duplicates
sorted_samples, sorted_logprobs ←
  SortDescending((samples, logprobs), by=logprobs)
sorted_samples, sorted_logprobs ←
  RemoveDuplicates((sorted_samples, sorted_logprobs))
// Keep the top N_preds samples
top_samples ← sorted_samples[: N_preds]
top_logprobs ← sorted_logprobs[: N_preds]
match_scores ← EmptyMatrix(N_c, N_preds)
for i ← 1 to N_preds do
    // Find BM25 scores between entire corpus and each sample
    bm25_scores ← BM25(top_samples[i], C)
    bm25_scores_scaled ← LinearScale(bm25_scores, axis=0)
    // Modulate by sequence score
    modulated_bm25_scores ←
      e^(ε·top_logprobs[i]) × bm25_scores_scaled
    match_scores[:,i] ← modulated_bm25_scores
candidate_scores ← MaxPool(match_scores, axis=1)    // size = N_c
return ArgSortDescending(candidate_scores)
```

We remove duplicate generations, sort them by sequence scores in descend order, and retain at most top N_{preds} unique predictions. Then, in order to match these generated texts with items from our corpus, we rely on BM25 retrieval [40] to calculate the matching scores between each text prediction and the whole item corpus (with size N_c): this will result in BM25 matching scores organized into a matrix of shape (N_c, N_{preds}). In order to rank the candidate items, we devise the following heuristics which aims to select the closest matches according to both the LLM sequence scores and BM25 scores in a quasi-round-robin fashion across the different N_{preds} text generations. First, we linearly scale each column of the matrix respectively to the range $[0, 1]$ to make BM25 scores derived from different LLM predictions comparable. We then penalize each prediction's BM25 scores (each column) by multiplying the BM25 scores by a factor of $e^{\epsilon s}$, where s is the sequence score associated with the text prediction, and ϵ is a small positive constant so that $e^{\epsilon s}$ is smaller than but very close to 1.0. This is to further modulate the BM25 text-based retrieval scores by the LLM generative scores. After that, we max-pool along the N_{preds} axis and finally derive modulated BM25 matching scores in a single vector of size N_c, with information from all N_{preds} recommendations fused in it (we can then derive the ranking of the ground-truth target item in the item corpus for calculating evaluation metric scores, or return top-K recommendations for users), as presented in Algorithm 1[3]. Here, we provide another view into the round-robin nature of our approach: given that the scaled BM25 score of each best-match

[3] In our preliminary investigation, we found that **1)** max-pooling significantly outperforms min-pooling and mean-pooling and **2)** there is obvious performance drop if we remove modulation since only max-pooling combined with modulation can make sure that top-1 LLM outputs' top-1 recommendation is still the top-1 recommendation in the merged list. **3)** The overall results are not sensitive to the value of ϵ as long as ϵ is small.

item from the non-duplicate N_{preds} texts generated is 1, when they are further modulated (scaled down) using $e^{\epsilon s}$, the final retrieval algorithm roughly follows a round-robin fashion, where the top-1 BM25 retrieval result of each of the generated text are in turn grouped as the final top N_{preds} retrieval results, when ϵ is close to 0.0. Since there are ties, we name our method 'quasi-' rather than 'strict-' round-robin.

4 Experimental Setup

LLM Selection. Our work leverages PaLM-2 [2, 46], a state-of-the-art LLM that is pretrained with a mixture of different objectives and exhibits exceptional generalization capabilities on a variety of language tasks. Among PaLM-2 model variants, we choose PaLM-2 XXS as the LLM backbone for our tasks, balancing model capability with inference latency.[4] Larger PaLM-2 S does not show a significant advantage against XXS model when both are without fine-tuning (see §5). Besides, the scale of our dataset (significantly smaller than text corpora for pretraining LLMs) may lead to overfitting when fine-tuning larger models [24, 34].

Dataset. Our experiments adopt the established and publicly available Amazon Review dataset 2018 [35]. Specifically, we use the 5-core subsets where each user/item is found in at least 5 interactions. We choose four item attributes: *title, category, brand*, and *price*, to construct the flattened text description of each item, and we truncate the length of their corresponding descriptions to 25, 15, 15, and 15 words, respectively. Following Li et al. [26], the same 14 product categories are involved in our research and they are all used for Stage I multi-category joint fine-tuning (3.59M users). Among them, 7 categories (3.37M users) are only for Stage I training where their entire user purchase records are used[5]; one category 'CD and Vinyls' is used for model selection purpose (29K users) during Stage I training where the last item for each user is left for validation and only the first $n-1$ items are adopted for training. Finally, the remaining 6 categories are adopted for category-specific fine-tuning (Stage II) and evaluation (0.22M Users)[6], where the first $n-2$ items are used for training in both Stages I and II, the penultimate item is used for validation in Stage II, and the very last item for evaluation. We also conduct data deduplication (dedup) on user sequences, which we describe later in next paragraph. For large categories including 'Office' and 'Pet', we randomly sampled 50% and 20% users respectively. The detailed statistics of our data in Stage II (after deduplication) are presented in Table 1, where 'Density' is defined as Total Purchase/(#of Users × #of Items), and the 'Word Vocab Size' denotes the lowercase natural word types over the item corpus spanning the four attributes used.

[4]Here, XXS is a 'T-shirt size' notation of the model size. S-, M- and L-sized PaLM-2 models reported in Anil et al. [2] are all larger than our XXS-sized model.

[5]The 7 categories are 'Automotive', 'Cell Phones and Accessories', 'Clothing Shoes and Jewelry ', 'Electronics', ' Grocery and Gourmet Food', 'Home and Kitchen', and 'Movies and TV'. The aim of leveraging large volume of data in Stage I is to benefit other categories in Stage II as a result of transfer learning.

[6]The 6 categories are 'Industrial and Scientific', 'Musical Instruments', 'Arts, Crafts and Sewing', 'Office Products', 'Video Games', 'Pet Supplies'. For brevity, we also denote them as 'Scientific', 'Instruments', 'Arts', 'Office', 'Games', and 'Pet', respectively.

User Sequence Deduplication. We have found that the Amazon Review data include a non-negligible percentage of consecutive duplicate events in user sequences, where the duplication is identified as two consecutive activities by the same user that have the same item ID (ASIN number), review text, rating, and the same review timestamp. As indicated in Table 1, around 4% to 9.4% of purchases are exact duplicates of the previous ones in their user sequences. We conduct deduplication (dedup) on all user sequences following our strict definition of duplication above, while we still keep two consecutive purchases of the same item, if they occurred at different times (i.e., they come with different timestamps) or come with different ratings/review text since they are likely the users' real actions. We note that some previous work, e.g. [26], did not conduct user sequence deduplication, which may lead a recommender system to learn a trivial strategy of always recommending the last item in the input sequence.

Implementation Details. We train our CALRec based on XXS-sized PaLM-2 on a cluster of TPUv4 chips [21]; our code is implemented with JAX [6][7] and PAX[8]. We adopt a training batch size of 512 users, a learning rate of 1e−4, a maximum input length of 1,024 tokens, and a maximum output decoding length of 80 tokens. As a data augmentation trick, during training, we randomly truncate the last k items ($k \in \{0, 1, 2, 3, 4\}$) from the full user record (with n items) and always use the last item after truncation (i.e., the $(n-k)$-th item in the original user record) as the target item. We train CALRec with $\alpha = 0.125$, $\beta = -0.025$ for up to 150,000 steps in Stage I and adopt $\tau_c = 0.5$ (see Eqs. 2 & 3) as the configuration yields best validation scores. We adopt the checkpoint with 135,000 training steps in Stage I selected on the validation data category, and then further fine-tune the model for Stage II. There are two exceptions: Scientific[9], and Games[10]. All our model selection is based on the NDCG@10 metric following previous work (specifically we adopt its pessimistic estimation which will be defined later in this section). For each case, the same training objective is used in Stages I and II. At inference, the temperature for decoding is 0.5, $N_{gen} = 32$, $N_{pred} = 10$ and $\epsilon = 1/5000$. There are two hyper-parameters inherent to BM25: we adopt the default values from the original BM25 API, $k_1 = 1.5$ and $b = 0.75$, throughout all our experiments and did not further tune them since we found that the model performance is not sensitive to them.

Baselines. In this work, we consider a comprehensive set of baseline approaches. SASRec [22] and BERT4Rec [45] are pure ID-Based approaches, Recformer [26] and the 'Last Item Repeater' (LIR) are purely text-based, and FDSA [57] and S^3-Rec [59] leverage both ID and text. UniSRec [18] has two variants and we report its performance in both text-only mode and ID+text mode. More details about each baseline are provided in what follows.[11]

[7]https://github.com/google/jax

[8]https://github.com/google/paxml

[9]We pick the Stage I model at 45,000 training steps since 'Scientific' is significantly smaller than other categories and its own validation set indicates that overfitting occurs at 135,000 steps and then continue with Stage II training.

[10]For Games category, we adopt $\alpha = 0.125, \beta = 0$ for both stages and its Stage I checkpoint at 105,000 steps is selected using the Stage I validation set. It is because we found on the 'Games' own validation set, the new setup outperforms the previous (default) setup.

[11]In our experiments, LIR is implemented on our own, Recformer follows its official implementation. The rest baselines are all provided by the official code repo of UniSRec

		Before Dedup	After Dedup						
Dataset	# of Users	% of duplicate Purchases	# of Items	Total Purchase	Items/User	Purchases/Item	Density	Words/Item	Word Vocab Size	Avg. Word Freq
Scientific	9461	5.5%	5282	66644	7.04	12.62	0.0013	22.82	19178	6.3
Instruments	25577	4.0%	10599	214526	8.39	20.24	0.00079	18.43	22256	8.8
Arts	47197	9.4%	22828	411449	8.72	18.02	0.00038	21.38	40342	12.1
Office	44736 (50%)	6.3%	27482	352151	7.87	12.81	0.00029	21.74	56687	10.5
Games	50940	4.7%	17383	457060	8.97	26.29	0.00051	16.39	17087	16.7
Pet	43135 (20%)	5.9%	37712	380623	8.82	10.09	0.00023	19.87	47339	15.8

Table 1: Statistics of Amazon Review dataset.

- **SASRec** [22] is a causal sequential recommendation approach implemented with an encoder-only unidirectional transformer. It takes in a sequence of item IDs and predicts the ID embedding of the next item in the last position.
- **BERT4Rec** [45] adopts a bidirectional transformer architecture for sequential recommendation. Its input consists of a sequence of item IDs and it is trained with the masked language modeling objective. At inference, a 'mask' token is appended to the end of the sequence to predict the next item.
- **FDSA** [57] applies two self-attentional blocks to address ID sequence and attribute feature sequence, respectively, and fuses their final representations for next-item prediction.
- **S^3-Rec** [59] maximizes the mutual information between ID and attribute embeddings via contrastive objectives and it is implemented with an encoder-only transformer network. It consists of a pretraining stage and a fine-tuning stage, both conducted on domain-specific data.
- **UniSRec** [18] separately encodes each item's text via a pretrained and fixed BERT cascaded by a trainable adapter. Given a sequence of item embeddings, a sequence representation is then modeled with another small transformer network. We adopt the officially released UniSRec model pretrained on multiple domains and fine-tune it on specific Amazon categories, respectively. We report two fine-tuning setups of UniSRec: (1) text-only setup, (2) ID+text setup where an item embedding is the sum of its ID and text representations.
- **Recformer** [26] relies on the encoder-only Longformer architecture [5] where token type and item position embeddings are proposed to facilitate learning item and user representations. It also features a pretraining stage on multiple categories with a combination of standard Masked Language Modeling (MLM) loss and a contrastive objective aligning user history and target item representations. We adopt Recformer's officially released pretrained checkpoint and fine-tune it on specific categories, respectively.
- **Last Item Repeater**, or **LIR** hereafter, denotes a simple yet effective baseline strategy which always recommends the last item in the input sequence as our training-free, text-statistic baseline. Based on the text of the last item, we rely on BM25 to rank the items in the full item corpus.

Evaluation Metrics. Following standard practices from prior work, we report NDCG@10, Recall@K (K$\in \{1, 10\}$) and MRR@∞ scores. Different from previous work, we report both 'optimistic' (*opt*) and 'pessimistic' (*pes*) calculations of the metrics: when there is a tie such that the ground-truth target item and other candidate items have the same matching score, the ground-truth item ranks first in (*opt*) and ranks last in (*pes*) calculation, respectively. It is because **1)** there are distinct items (with different item IDs) with the same text description and **2)** in rare cases, we found that BM25 may yield identical scores for very similar text pieces. We still treat items in case **1)** as distinct items because in real production environments vendors/content creators may intentionally or unintentionally duplicate items. Of course, this will not cause any issue for ID-based approaches due to distinct item IDs, where (*opt*) and (*pes*) scores are the same. In the official implementations of our text-based baselines, Li et al. [26] only calculates (*opt*) metric scores which may result in unfair comparisons with ID-based baselines. Hou et al. [18] applies "`torch.topk()`" to derive exactly top K item ids for calculating Recall/NDCG@K: if the K-th and the $K+1$-th items are having the same score, only one of them will be included in the output list. So its metric score is a number between our (*opt*) and (*pes*) estimates.[12] All our evaluations are performed on user-sequence-deduplicated data.

which is based on RecBole [58]: the attribute embeddings of FDSA and S^3-Rec are text features from our strongest baseline UniSRec.

5 Experimental Results

Main Results. Table 2 demonstrates that the sequential recommendation capabilities of the proposed CALRec framework are stronger than our baselines. On the 6 Amazon Review test categories, our model outperforms all 8 baselines significantly in terms of NDCG@10, Recall@1 and MRR. In terms of Recall@10, our approach outperforms all Text-Only and ID-Only models across the board by a large margin (including the Text-Only variant of UniSRec), but slightly underperforms our strongest baseline, the ID+Text variant of UniSRec. For our method and 3 Text-Only baselines, the results in Table 2 verified that there is a gap between optimistic and pessimistic metric scores, demonstrating the necessity of reporting both. Meanwhile, it is worth noticing that even our pessimistic scores clearly surpass the optimistic scores of the 3 Text-Only baseline models on all the 6 data categories. In addition to the absolute scores, we also calculate the percentage gains comparing CALRec's optimistic scores against the baseline SotA for each data category and each evaluation metric (different method for different data category and metric). The results are presented in the last column of Table 2. We found that, CALRec is especially strong

[12]To provide more insights on our (*opt*)/(*pes*) metrics, readers could consider an extreme scenario where the model output is empty and thus when using BM25 retrieval the matching score is 0 for all candidate items. In this case, the optimistic Recall@K scores will always be 1.0, while the pessimistic scores will always be 0.0. To provide a fair assessment of the model's ability to uniquely identify candidate items, we have thus decided to report both (*opt*) and (*pes*) scores.

Dataset	Metric (*opt*)/(*pes*)	ID-Only Methods		ID+Text Methods			Text-Only Methods				Improv. (*opt*)
		BERT4Rec	SASRec	FDSA	S^3-Rec	UniSRec	UniSRec	RecFormer	LIR	CALRec	
Scientific	NDCG@10	0.0411	0.0555	0.0594	0.0414	0.0644	0.0620/0.0580	0.0639/0.0588	0.0565/0.0486	**0.0788**/0.0740	22.4%
	Recall@1	0.0239	0.0079	0.0357	0.0172	0.0199	0.0242/0.0140	0.0298/0.0168	0.0142/0.0052	**0.0511**/0.0389	43.1%
	Recall@10	0.0636	0.1065	0.0887	0.0727	**0.1181**	0.1101/0.1100	0.1058/0.1057	0.1005/0.0933	0.1124/0.1117	-4.8%
	MRR	0.0389	0.0456	0.0555	0.0380	0.0555	0.0546/0.0492	0.0570/0.0502	0.0461/0.0385	**0.0730**/0.0667	28.1%
Instruments	NDCG@10	0.0680	0.0623	0.0796	0.0606	0.0709	0.0661/0.0655	0.0596/0.0585	0.0344/0.0338	**0.0909**/0.0905	14.2%
	Recall@1	0.0494	0.0158	0.0574	0.0202	0.0237	0.0251/0.0236	0.0266/0.0239	0.0040/0.0031	**0.0718**/0.0706	25.1%
	Recall@10	0.0915	0.1130	0.1092	0.1048	**0.1255**	0.1141/0.1141	0.0940/0.0940	0.0696/0.0692	0.1158/0.1158	-7.7%
	MRR	0.0658	0.0528	0.0765	0.0526	0.0613	0.0576/0.0569	0.0535/0.0521	0.0263/0.0257	**0.0864**/0.0857	12.9%
Arts	NDCG@10	0.0547	0.0619	0.0726	0.0602	0.0729	0.0630/0.0624	0.0662/0.0653	0.0443/0.0431	**0.0864**/0.0853	18.5%
	Recall@1	0.0347	0.0147	0.0460	0.0216	0.0213	0.0220/0.0208	0.0279/0.0254	0.0067/0.0051	**0.0636**/0.0610	38.3%
	Recall@10	0.0799	0.1118	0.1061	0.1052	**0.1320**	0.1103/0.1102	0.1095/0.1095	0.0859/0.0850	0.1140/0.1139	-13.6%
	MRR	0.0513	0.0519	0.0680	0.0520	0.0615	0.0546/0.0540	0.0582/0.0569	0.0344/0.0331	**0.0815**/0.0801	19.9%
Office	NDCG@10	0.0545	0.0602	0.0749	0.0607	0.0713	0.0625/0.0616	0.0687/0.0676	0.0391/0.0379	**0.0976**/0.0966	30.3%
	Recall@1	0.0403	0.0134	0.0547	0.0291	0.0213	0.0250/0.0228	0.0328/0.0302	0.0056/0.0040	**0.0761**/0.0734	39.1%
	Recall@10	0.0709	0.1033	0.0982	0.0941	**0.1220**	0.1023/0.1022	0.1063/0.1063	0.0735/0.0728	0.1213/0.1213	-0.6%
	MRR	0.0520	0.0499	0.0710	0.0535	0.0597	0.0545/0.0533	0.0603/0.0588	0.0309/0.0296	**0.0925**/0.0911	30.3%
Games	NDCG@10	0.0334	0.0451	0.0526	0.0419	0.0501	0.0408/0.0407	0.0377/0.0360	0.0211/0.0197	**0.0595**/0.0585	13.1%
	Recall@1	0.0112	0.0039	0.0210	0.0080	0.0075	0.0111/0.0107	0.0126/0.0091	0.0015/0.0009	**0.0295**/0.0277	40.5%
	Recall@10	0.0646	0.0989	0.0957	0.0884	**0.1074**	0.0820/0.0820	0.0724/0.0723	0.0442/0.0427	0.0986/0.0984	-8.2%
	MRR	0.0313	0.0380	0.0483	0.0367	0.0427	0.0372/0.0370	0.0349/0.0326	0.0171/0.0159	**0.0510**/0.0498	5.6%
Pet	NDCG@10	0.0376	0.0448	0.0537	0.0392	0.0574	0.0539/0.0464	0.0590/0.0471	0.0391/0.0354	**0.0736**/0.0693	24.8%
	Recall@1	0.0276	0.0093	0.0394	0.0162	0.0202	0.0295/0.0129	0.0377/0.0130	0.0094/0.0030	**0.0570**/0.0465	44.7%
	Recall@10	0.0499	0.0773	0.0710	0.0647	**0.0970**	0.0829/0.0826	0.0826/0.0818	0.0680/0.0673	0.0937/0.0934	-3.4%
	MRR	0.0365	0.0382	0.0522	0.0349	0.0503	0.0498/0.0399	0.0549/0.0393	0.0317/0.0272	**0.0696**/0.0639	26.8%
Average (6 Categories)	NDCG@10	0.0482	0.0550	0.0655	0.0507	0.0645	0.0581/0.0558	0.0592/0.0556	0.0391/0.0364	**0.0811**/0.0790	23.8%
	Recall@1	0.0312	0.0108	0.0424	0.0187	0.0190	0.0228/0.0175	0.0279/0.0197	0.0069/0.0036	**0.0582**/0.0530	37.3%
	Recall@10	0.0701	0.1018	0.0948	0.0883	**0.1170**	0.1003/0.1002	0.0951/0.0949	0.0736/0.0717	0.1093/0.1091	-6.6%
	MRR	0.0460	0.0461	0.0619	0.0446	0.0552	0.0514/0.0484	0.0531/0.0483	0.0311/0.0283	**0.0757**/0.0729	22.3%

Table 2: Evaluation results of CALRec in comparison with different ID/Text-based sequential recommendation baselines (cf. §4). For text-only methods, we report both optimisitc (*opt*) and pessimistic (*pes*) as '(*opt*)/(*pes*)'. For ID-based methods, (*opt*) and (*pes*) scores are the same. Bold: the highest (*opt*) scores. Underline: the strongest (*pes*) scores. We also report the relative improvement (*opt*) comparing CALRec against (*opt*) baseline SotA marked with Blue Background per data category per metric. Evaluation metrics are computed using the Amazon Review dataset after user sequence deduplication (see also §4).

Model Variant	Scientific				Instruments				Games			
	NDCG@10	Recall@1	Recall@10	MRR	NDCG@10	Recall@1	Recall@10	MRR	NDCG@10	Recall@1	Recall@10	MRR
CALRec	**0.0788** / 0.0740	**0.0511** / 0.0389	**0.1124** / 0.1117	**0.0730** / 0.0667	**0.0909** / 0.0905	**0.0718** / 0.0706	**0.1158** / 0.1158	**0.0864** / 0.0857	**0.0595** / 0.0585	**0.0295** / 0.0277	0.0986 / 0.0984	**0.0510** / 0.0498
w/o Multi-Category Joint Fine-Tuning (A)	0.0727 / 0.0675	0.04408 / 0.0314	0.1070 / 0.1061	0.0670 / 0.0602	0.0870 / 0.0862	0.0673 / 0.0658	0.1119 / 0.1115	0.0822 / 0.0812	0.0583 / 0.0570	0.0286 / 0.0265	0.0966 / 0.0959	0.0501 / 0.0486
w/o Category-Specific Fine-Tuning (B)	0.0717 / 0.0670	0.0424 / 0.0300	0.1077 / 0.1073	0.0653 / 0.0588	0.0867 / 0.0862	0.0715 / 0.0702	0.1048 / 0.1048	0.0840 / 0.0834	0.0539 / 0.0530	0.0257 / 0.0238	0.0904 / 0.0902	0.0463 / 0.0451
w/o Contrastive Alignment	0.0771 / 0.0724	0.0495 / 0.0376	0.1109 / 0.1105	0.0712 / 0.0649	0.0905 / 0.0899	0.0711 / 0.0696	0.1154 / 0.1153	0.0858 / 0.0850	0.0593 / 0.0584	0.0289 / 0.0272	**0.0987** / 0.0985	0.0507 / 0.0495
Pretrained PaLM-2 (w/o Fine-Tuning)	0.0326 / 0.0291	0.0090 / 0.0055	0.0661 / 0.0623	0.0267 / 0.0234	0.0257 / 0.0254	0.0041 / 0.0037	0.0585 / 0.0584	0.0197 / 0.0193	0.0150 / 0.0141	0.0031 / 0.0024	0.0325 / 0.0314	0.0134 / 0.0127

Table 3: Ablation result showing the benefits of multi-stage fine-tuning and constrastive alignment to improve the performance of CALRec. Bold: the highest optimistic (*opt*) scores. Underline: the strongest pessimistic (*pes*) scores.

in Recall@1, outperforming baseline SotA by 37.3% on average; CALRec outperforms baseline SotA by 23.8% and 22.3% on average in terms of NDCG@10 and MRR, respectively.

Ablation Study. In Table 3, we demonstrate the effectiveness of CALRec's key components via extensive ablation study on 3 data categories (the results on the other 3 categories show the same trends so we skip them due to space constraints).

Two-Stage Fine-Tuning: First, it is obvious that our Stage I multi-category joint fine-tuning and Stage II category-specific fine-tuning are both effective, without which there will be an obvious drop in performance. Furthermore, it is worth mentioning that our model derived only with Stage I is able to recommend items in all 6 categories: this is especially valuable and easy to deploy in real-world applications, due to its multi-tenancy nature. Although the multi-category trained model (variant B in Table 3) lags behind the full

Metric: (*opt*)/(*pes*)	w/ Contr.	w/o Contr.	Improv.	*p*-value
NDCG@10	0.0811/0.0790	0.0803/0.0782	1.00%/1.02%	0.0076/0.0064
Recall@1	0.0582/0.0530	0.0574/0.0521	1.39%/1.73%	0.0027/0.0004
Recall@10	0.1093/0.1091	0.1084/0.1082	0.83%/0.83%	0.0178/0.019
MRR	0.0757/0.0729	0.0748/0.0720	1.20%/1.25%	0.0066/0.0037

Table 4: Average scores (*opt*)/(*pes*) on all 6 data categories: w/ contrastive alignment vs. w/o contrastive alignment.

Dataset	Metric (*opt*)/(*pes*)	Model Size		
		XXXS	XXS	S
Scientific	NDCG@10	0.0299/0.0252	**0.0326**/0.0291	0.0321/0.0306
	Recall@1	0.0089/0.0034	**0.0090**/0.0055	0.0082/0.0058
	Recall@10	0.0592/0.0534	**0.0661**/0.0623	0.0660/0.0647
	MRR	0.0246/0.0204	**0.0267**/0.0234	0.0262/0.0247
Instruments	NDCG@10	0.0215/0.0210	**0.0257**/0.0254	0.0234/0.0231
	Recall@1	0.0029/0.0022	0.0041/0.0037	**0.0043**/0.0039
	Recall@10	0.0480/0.0476	**0.0585**/0.0584	0.0512/0.0511
	MRR	0.0162/0.0156	**0.0197**/0.0193	0.0188/0.0185

Table 5: Performance results of pretrained PaLM-2 models of three distinct sizes without any fine-tuning for Sequential Recommendation. Bold: the highest optimistic (*opt*) scores. Underline: the strongest pessimistic (*pes*) scores.

Dataset	Metric	(*opt*)/(*pes*)
Scientific	NDCG@10	0.0265/0.0232
	Recall@1	0.0051/0.0030
	Recall@10	0.0582/0.0537
	MRR	0.0217/0.0189
Instruments	NDCG@10	0.0240/0.0237
	Recall@1	0.0028/0.0025
	Recall@10	0.0584/ 0.0583
	MRR	0.0179/0.0175

Table 6: Performance results of GPT4Rec's template using pretrained PaLM-2 XXS without any fine-tuning for Sequential Recommendation.

CALRec derived with both stages, it is already on par with variant A models (cf. Table 3). We also compare CALRec and pretrained 'off-the-shelf' PaLM-2 XXS in Table 3; the results show that without any fine-tuning, PaLM-2 demonstrates poor performance on our sequential recommendation benchmarks.

Contrastive Objectives: In Table 4 we calculate the average scores on all 6 categories for the setups with and without the auxiliary contrastive losses. Basically, the auxiliary contrastive objective can result in circa $0.8\% - 1.7\%$ gains. We also conduct dependent paired sample t-test [41] and report the p-values in the last column of Table 4: the gains are all statistically significant at 0.05 level.

Model Size: We are also interested in how pretrained PaLM-2 *without* any fine-tuning on our sequential recommendation data performs in our tasks and we present results with three PaLM-2 models of different model sizes (i.e., XXXS, XXS, and S) on 'Scientific' and 'Instruments' categories in Table 5. The input/output format is still the same: input data is from a single user. The results show that XXS and S models actually achieve similar performance and they both outperform the XXXS model.

Template Design: We conducted additional experiments on prompting the raw PaLM2 XXS model for recommendation adopting another prompt from GPT4Rec [27], with results reported in Table 6. Unlike our prompt template, their template doesn't include a shared per-item prefix and user suffix but leverages a one-off user suffix. Compared with our results reported in Table 5 with the same raw PaLM-2 XXS, the new results are universally lower than ours (especially recall@1, relatively $30\% \sim 45\%$ drop in scores), showing the effectiveness of our template design.

6 Further Analyses and Discussion

Further Discussion on User Sequence Dedup. Now, we further delve deeper into the issue of duplicate interactions in user sequences. Although in real world users can make repeated activities, we still posit that the high ratio of consecutive identical interactions in the raw Amazon Review data are not likely to reflect normal/real user behavior (further details in Appendix/Auxiliary Materials). While we *cannot conclude* that data deduplication is 'correct' or 'wrong', we reckon that the 'LIR' baseline results are meaningful and should be reported for model comparison purposes. We further run the LIR Baseline on **1)** data from Li et al. [26] without dedup and compare with its results on **2)** our data with dedup. Although the data statistics are slightly different, the comparisons are still meaningful, since their data sizes are comparable and they are both representative of the full distribution. Table 7 shows the optimistic metrics of LIR Baseline that always recommends last item in the input sequence on data w/o dedup (left four columns) and data w/ dedup (right four columns). All metrics suffer sharp declines due to the dedup process. Specifically, Recall@1 drops 60%-95% after removing consecutive identical events. This highlights the importance of user sequence interaction deduplication as a critical step for reliably measuring model quality.

Does CALRec Rely on BM25? One challenge of using generative LLMs, especially for general-purpose LLMs not tuned with task-specific data, for recommendation is that there is no guarantee that the predicted item exists in the item corpus. Therefore we use BM25, a fuzzy lexical matching algorithm, to rank the items in the item corpus. However, the fine-tuned CALRec models can quickly master the item template format and remember items seen during training, making direct text-entity based retrieval possible without relying on fuzzy text matching methods. This motivates a simple and faster alternative to BM25.

First, we found that nearly all ($> 99.9\%$) fine-tuned model outputs adhere to the desired item text format: "Title: **. Category: **. Brand: **. Price: **.". We then extract the four attribute entities: title, category, brand, and price, and find that $82\% - 98\%$ of model outputs correspond to an item in the item corpus that exactly matches all four attributes, and $95\% - 99.8\%$ of title entities extracted from the model outputs exist in the item corpus. These statistics are summarized in Table 8. Based on these findings, we propose a heuristic hierarchical matching method. We first search exact matches in the item corpus for all four attributes extracted from model outputs; If

	w/o Dedup - Metric: (*opt*)				w/ Dedup - Metric: (*opt*)			
Dataset	NDCG@10	Recall@1	Recall@10	MRR	NDCG@10	Recall@1	Recall@10	MRR
Scientific	0.088	0.0355	0.1381	0.0745	0.0565 (-36%)	0.0142 (-60%)	0.1005 (-27%)	0.0461 (-38%)
Instruments	0.0545	0.0301	0.081	0.0485	0.0344 (-37%)	0.0040 (-87%)	0.0696 (-14%)	0.0263 (-46%)
Arts	0.0946	0.0511	0.1464	0.0811	0.0443 (-53%)	0.0067 (-87%)	0.0859 (-41%)	0.0344 (-58%)
Office	0.0757	0.0424	0.1095	0.0669	0.0391 (-48%)	0.0056 (-87%)	0.0735 (-33%)	0.0309 (-54%)
Games	0.05	0.0275	0.0755	0.0444	0.0211 (-58%)	0.0015 (-95%)	0.0442 (-41%)	0.0171 (-61%)
Pet	0.069	0.0318	0.1085	0.0578	0.0391 (-43%)	0.0094 (-70%)	0.0680 (-37%)	0.0317 (-45%)

Table 7: Data deduplication. Optimistic evaluations of LIR Baseline (always recommend last-item in the input sequence) on data from [26] (w/o dedup) and our data w/ dedup respectively.

		Output Matches an Item (%)		Hierarchical Match - Metric: (*opt*)/(*pes*) & Improv. vs BM25 (%)			
Dataset	Format Error (%)	All Four Attributes	Title Only	NDCG@10	Recall@1	Recall@10	MRR
Scientific	0.0%	82.3%	96.7%	0.0771 / 0.0725 (-2.1%)	0.0525 / 0.0404 (+3.2%)	0.1054 / 0.1054 (-5.9%)	0.0693 / 0.0632 (-5.2%)
Instruments	0.001%	97.6%	99.8%	0.0905 / 0.0901 (-0.4%)	0.0717 / 0.0705 (-0.1%)	0.1151 / 0.1151 (-0.6%)	0.0836 / 0.0830 (-3.2%)
Arts	0.01%	92.5%	98.9%	0.0854 / 0.0844 (-1.1%)	0.0636 / 0.0611 (+0.1%)	0.1113 / 0.1113 (-2.3%)	0.0777 / 0.0764 (-4.6%)
Office	0.0%	84.5%	94.7%	0.0926 / 0.0916 (-5.2%)	0.0736 / 0.0710 (-3.3%)	0.1127 / 0.1127 (-7.1%)	0.0864 / 0.0851 (-6.6%)
Games	0.05%	98.0%	99.0%	0.0582 / 0.0573 (-2.1%)	0.0276 / 0.0257 (-6.8%)	0.0973 / 0.0972 (-1.3%)	0.0467 / 0.0456 (-8.4%)
Pet	0.03%	97.3%	99.0%	0.0733 / 0.0690 (-0.4%)	0.0573 / 0.0468 (+0.6%)	0.0924 / 0.0922 (-1.3%)	0.0677 / 0.0620 (-2.8%)

Table 8: Hierarchical match results. The relative gains/drops in $\frac{(opt)+(pes)}{2}$ are also reported against BM25 retrieval.

no match is found, we search for the exact match of the first three attributes, then the first two, and finally only the title description. Similar to our main experiments, both optimistic and pessimistic metrics are calculated, with results also presented in Table 8: we find that our hierarchical matching method has marginally worse performance compared to our BM25-based retrieval, yet it still outperforms other baselines models presented in Table 2.

Limitations and Future Work. Due to the fact that fine-tuned CALRec usually outputs exact item descriptions seen during training, a direct limitation ensues: CALRec barely cannot address the *item cold-start* scenario where the ground-truth item is neither covered in the training set nor in the validation set. We leave addressing this limitation to future work. While previous approaches that also rely on matching LLM-generated item descriptions (e.g., titles) did not provide experimental results on item cold-start cases [20, 27], we emphasize that our work is the first to identify this issue, which may also be present in these and similar approaches (yet to be verified). We regard this finding a minor contribution of our study.

Second, although CALRec outperforms other baselines in Recall@1, NDCG@10, and MRR@∞ by large margin, its Recall@10 underperforms one of our ID+text baselines. We attribute this to the quasi-greedy nature of temperature sampling (especially when adopting a relatively small temperature value). Unlike embedding-based nearest neighbor search, our approach may not consistently produce the most optimal top-10 item predictions from the entire item corpus. Consequently, although CALRec excels in metrics that emphasize the relevance of the very top item prediction (like Recall@1 and MRR), there is a need for greater diversity in its top-10 item predictions.

Third, in real-world scenarios, when the size of item corpus is extremely large (e.g., billions of items), BM25 retrieval becomes prohibitively slow and often infeasible. In addition to the hierachical entity matching method discussed earlier in this section, we may leverage CALRec as a reranker within a retrieve-and-rerank framework where any baseline sequential recommendation approach can be used to first derive, for example, top 10K recommendations focused on high recall. Then CALRec's output texts are used to rerank these items. This retrieve-and-rerank paradigm can not only leverage the strengths of CALRec but also circumvents the need for BM25 retrieval on a massive corpus.

7 Conclusion

We proposed CALRec, a contrastive-learning-assisted two-stage training framework for sequential recommendation based on LLMs, with experiments conducted using the PaLM-2 LLM as the backbone. Our approach also features careful template design inspired from few-shot learning and a novel quasi-round-robin BM25 retrieval approach. Comprehensive experiments on the Amazon Review dataset demonstrate the effectiveness of CALRec, supported by PaLM-2, where it significantly outperforms existing SotA models for sequential recommendation by a considerable margin in most evaluation metrics. We also presented a series of further analyses and revisited the issue of duplicate interactions in user sequences and existing evaluation metrics; we accordingly propose to adopt a training-free text-statistic based 'Last Item Repeater' reference baseline for model comparison purposes, and we strongly suggest reporting both optimistic and pessimistic metric scores for all text-based approaches.

Acknowledgments

We thank the following Google colleagues (in alphabetical order) for their technical advice and helpful discussions on this work: Aahil Mehta, Ao Liu, Boaz Cogan, Jianmo Ni, Minh Pham, Nicolas Aagnes, Wang-Cheng Kang, Xin Zhang, Yan Zhu. We also extend our gratitude to anonymous reviewers for their constructive feedback.

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023). https://doi.org/10.48550/arXiv.2303.08774

[2] Rohan Anil, Andrew M Dai, Orhan Firat, Melvin Johnson, Dmitry Lepikhin, Alexandre Passos, Siamak Shakeri, Emanuel Taropa, Paige Bailey, Zhifeng Chen, et al. 2023. Palm 2 technical report. *arXiv preprint arXiv:2305.10403* (2023). https://doi.org/10.48550/arXiv.2305.10403

[3] Anurag Arnab, Mostafa Dehghani, Georg Heigold, Chen Sun, Mario Lučić, and Cordelia Schmid. 2021. ViViT: A Video Vision Transformer. In *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*. 6816–6826. https://doi.org/10.1109/ICCV48922.2021.00676

[4] Keqin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. 2023. TALLRec: An Effective and Efficient Tuning Framework to Align Large Language Model with Recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems* (Singapore, Singapore) *(RecSys '23)*. Association for Computing Machinery, New York, NY, USA, 1007–1014. https://doi.org/10.1145/3604915.3608857

[5] Iz Beltagy, Matthew E Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150* (2020). https://doi.org/10.48550/arXiv.2004.05150

[6] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. 2018. *JAX: composable transformations of Python+NumPy programs*. http://github.com/google/jax

[7] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901. https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf

[8] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. 2020. A Simple Framework for Contrastive Learning of Visual Representations. In *Proceedings of the 37th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 119)*, Hal Daumé III and Aarti Singh (Eds.). PMLR, 1597–1607. https://proceedings.mlr.press/v119/chen20j.html

[9] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2023. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research* 24, 240 (2023), 1–113. http://jmlr.org/papers/v24/22-1144.html

[10] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, Jill Burstein, Christy Doran, and Thamar Solorio (Eds.). Association for Computational Linguistics, Minneapolis, Minnesota, 4171–4186. https://doi.org/10.18653/v1/N19-1423

[11] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. https://openreview.net/forum?id=YicbFdNTTy

[12] Angela Fan, Mike Lewis, and Yann Dauphin. 2018. Hierarchical Neural Story Generation. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Iryna Gurevych and Yusuke Miyao (Eds.). Association for Computational Linguistics, Melbourne, Australia, 889–898. https://doi.org/10.18653/v1/P18-1082

[13] Tianyu Gao, Xingcheng Yao, and Danqi Chen. 2021. SimCSE: Simple Contrastive Learning of Sentence Embeddings. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.). Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 6894–6910. https://doi.org/10.18653/v1/2021.emnlp-main.552

[14] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. 2022. Recommendation as Language Processing (RLP): A Unified Pretrain, Personalized Prompt & Predict Paradigm (P5). In *Proceedings of the 16th ACM Conference on Recommender Systems (RecSys '22)*. Association for Computing Machinery, New York, NY, USA, 299–315. https://doi.org/10.1145/3523227.3546767

[15] David Goldberg, David Nichols, Brian M. Oki, and Douglas Terry. 1992. Using collaborative filtering to weave an information tapestry. *Commun. ACM* 35, 12 (dec 1992), 61–70. https://doi.org/10.1145/138859.138867

[16] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2016. Session-based Recommendations with Recurrent Neural Networks. In *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, Yoshua Bengio and Yann LeCun (Eds.). http://arxiv.org/abs/1511.06939

[17] Yupeng Hou, Zhankui He, Julian McAuley, and Wayne Xin Zhao. 2023. Learning Vector-Quantized Item Representation for Transferable Sequential Recommenders. In *Proceedings of the ACM Web Conference 2023 (WWW '23)*. Association for Computing Machinery, New York, NY, USA, 1162–1171. https://doi.org/10.1145/3543507.3583434

[18] Yupeng Hou, Shanlei Mu, Wayne Xin Zhao, Yaliang Li, Bolin Ding, and Ji-Rong Wen. 2022. Towards Universal Sequence Representation Learning for Recommender Systems. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Washington DC, USA) *(KDD '22)*. Association for Computing Machinery, New York, NY, USA, 585–593. https://doi.org/10.1145/3534678.3539381

[19] Yupeng Hou, Junjie Zhang, Zihan Lin, Hongyu Lu, Ruobing Xie, Julian McAuley, and Wayne Xin Zhao. 2024. Large Language Models are Zero-Shot Rankers for Recommender Systems. In *Advances in Information Retrieval: 46th European Conference on Information Retrieval, ECIR 2024, Glasgow, UK, March 24–28, 2024, Proceedings, Part II* (Glasgow, United Kingdom). Springer-Verlag, Berlin, Heidelberg, 364–381. https://doi.org/10.1007/978-3-031-56060-6_24

[20] Jianchao Ji, Zelong Li, Shuyuan Xu, Wenyue Hua, Yingqiang Ge, Juntao Tan, and Yongfeng Zhang. 2024. GenRec: Large Language Model for Generative Recommendation. In *Advances in Information Retrieval: 46th European Conference on Information Retrieval, ECIR 2024, Glasgow, UK, March 24–28, 2024, Proceedings, Part III* (Glasgow, United Kingdom). Springer-Verlag, Berlin, Heidelberg, 494–502. https://doi.org/10.1007/978-3-031-56063-7_42

[21] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, et al. 2023. Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–14. https://doi.org/10.1145/3579371.3589350

[22] Wang-Cheng Kang and Julian McAuley. 2018. Self-Attentive Sequential Recommendation. In *2018 IEEE International Conference on Data Mining (ICDM)*. 197–206. https://doi.org/10.1109/ICDM.2018.00035

[23] Wang-Cheng Kang, Jianmo Ni, Nikhil Mehta, Maheswaran Sathiamoorthy, Lichan Hong, Ed Chi, and Derek Zhiyuan Cheng. 2023. Do llms understand user preferences? evaluating llms on user rating prediction. *arXiv preprint arXiv:2305.06474* (2023). https://doi.org/10.48550/arXiv.2305.06474

[24] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. 2017. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences* 114, 13 (2017), 3521–3526. https://doi.org/10.1073/pnas.1611835114

[25] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix Factorization Techniques for Recommender Systems. *Computer* 42, 8 (2009), 30–37. https://doi.org/10.1109/MC.2009.263

[26] Jiacheng Li, Ming Wang, Jin Li, Jinmiao Fu, Xin Shen, Jingbo Shang, and Julian McAuley. 2023. Text Is All You Need: Learning Language Representations for Sequential Recommendation. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. Association for Computing Machinery, New York, NY, USA, 1258–1267. https://doi.org/10.1145/3580305.3599519

[27] Jinming Li, Wentao Zhang, Tian Wang, Guanglei Xiong, Alan Lu, and Gerard Medioni. 2023. GPT4Rec: A generative framework for personalized recommendation and user interests interpretation. *arXiv preprint arXiv:2304.03879* (2023). https://doi.org/10.48550/arXiv.2304.03879

[28] Yaoyiran Li, Ching-Yun Chang, Stephen Rawls, Ivan Vulić, and Anna Korhonen. 2023. Translation-Enhanced Multilingual Text-to-Image Generation. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 9174–9193. https://doi.org/10.18653/v1/2023.acl-long.510

[29] Yaoyiran Li, Anna Korhonen, and Ivan Vulić. 2023. On Bilingual Lexicon Induction with Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 9577–9599. https://doi.org/10.18653/v1/2023.emnlp-main.595

[30] Yaoyiran Li, Anna Korhonen, and Ivan Vulić. 2024. Self-Augmented In-Context Learning for Unsupervised Word Translation. *arXiv preprint arXiv:2402.10024* (2024). https://doi.org/10.48550/arXiv.2402.10024

[31] Yaoyiran Li, Fangyu Liu, Nigel Collier, Anna Korhonen, and Ivan Vulić. 2022. Improving Word Translation via Two-Stage Contrastive Learning. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.). Association for Computational Linguistics, Dublin, Ireland, 4353–4374. https://doi.org/10.18653/v1/2022.acl-long.299

[32] Jianghao Lin, Xinyi Dai, Yunjia Xi, Weiwen Liu, Bo Chen, Xiangyang Li, Chenxu Zhu, Huifeng Guo, Yong Yu, Ruiming Tang, et al. 2023. How can recommender systems benefit from large language models: A survey. *arXiv preprint arXiv:2306.05817* (2023). https://doi.org/10.48550/arXiv.2306.05817

[33] Junling Liu, Chao Liu, Peilin Zhou, Renjie Lv, Kang Zhou, and Yan Zhang. 2023. Is ChatGPT a Good Recommender? A Preliminary Study. *arXiv preprint arXiv:2304.10149* (2023). https://doi.org/10.48550/arXiv.2304.10149

[34] Eric Mitchell, Charles Lin, Antoine Bosselut, Chelsea Finn, and Christopher D Manning. 2022. Fast Model Editing at Scale. In *International Conference on Learning Representations*. https://openreview.net/pdf?id=0DcZxeWfOPt

[35] Jianmo Ni, Jiacheng Li, and Julian McAuley. 2019. Justifying Recommendations using Distantly-Labeled Reviews and Fine-Grained Aspects. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan (Eds.). Association for Computational Linguistics, Hong Kong, China, 188–197. https://doi.org/10.18653/v1/D19-1018

[36] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748* (2018). https://doi.org/10.48550/arXiv.1807.03748

[37] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.* 21, 1, Article 140 (jan 2020), 67 pages. http://jmlr.org/papers/v21/20-074.html

[38] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q. Tran, Jonah Samost, Maciej Kula, Ed H. Chi, and Maheswaran Sathiamoorthy. 2023. Recommender Systems with Generative Retrieval. In *Thirty-seventh Conference on Neural Information Processing Systems*. https://openreview.net/forum?id=BJ0fQUU32w

[39] Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. 2022. Hierarchical text-conditional image generation with clip latents. *arXiv preprint arXiv:2204.06125* (2022). https://doi.org/10.48550/arXiv.2204.06125

[40] Stephen Robertson, Hugo Zaragoza, et al. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval* 3, 4 (2009), 333–389. https://doi.org/10.1561/1500000019

[41] Amanda Ross and Victor L. Willson. 2017. *Paired Samples T-Test*. SensePublishers, Rotterdam, 17–19. https://doi.org/10.1007/978-94-6351-086-8_4

[42] Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. 2001. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th International Conference on World Wide Web* (Hong Kong, Hong Kong) *(WWW '01)*. Association for Computing Machinery, New York, NY, USA, 285–295. https://doi.org/10.1145/371920.372071

[43] Yue Shi, Martha Larson, and Alan Hanjalic. 2014. Collaborative Filtering beyond the User-Item Matrix: A Survey of the State of the Art and Future Challenges. *ACM Comput. Surv.* 47, 1, Article 3 (may 2014), 45 pages. https://doi.org/10.1145/2556270

[44] Anima Singh, Trung Vu, Raghunandan Keshavan, Nikhil Mehta, Xinyang Yi, Lichan Hong, Lukasz Heldt, Li Wei, Ed Chi, and Maheswaran Sathiamoorthy. 2023. Better Generalization with Semantic IDs: A case study in Ranking for Recommendations. *arXiv preprint arXiv:2306.08121* (2023). https://doi.org/10.48550/arXiv.2306.08121

[45] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management* (Beijing, China) *(CIKM '19)*. Association for Computing Machinery, New York, NY, USA, 1441–1450. https://doi.org/10.1145/3357384.3357895

[46] Yi Tay, Mostafa Dehghani, Vinh Q. Tran, Xavier Garcia, Jason Wei, Xuezhi Wang, Hyung Won Chung, Dara Bahri, Tal Schuster, Steven Zheng, Denny Zhou, Neil Houlsby, and Donald Metzler. 2023. UL2: Unifying Language Learning Paradigms. In *The Eleventh International Conference on Learning Representations*. https://openreview.net/forum?id=6ruVLB727MC

[47] Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805* (2023). https://doi.org/10.48550/arXiv.2312.11805

[48] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. *arXiv preprint arXiv:2302.13971* (2023). https://doi.org/10.48550/arXiv.2302.13971

[49] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

[50] Lei Wang and Ee-Peng Lim. 2023. Zero-Shot Next-Item Recommendation using Large Pretrained Language Models. *arXiv preprint arXiv:2304.03153* (2023). https://doi.org/10.48550/arXiv.2304.03153

[51] Chenxi Whitehouse, Fantine Huot, Jasmijn Bastings, Mostafa Dehghani, Chu-Cheng Lin, and Mirella Lapata. 2023. Low-Rank Adaptation for Multilingual Summarization: An Empirical Study. *arXiv preprint arXiv:2311.08572* (2023). https://doi.org/10.48550/arXiv.2311.08572

[52] Likang Wu, Zhi Zheng, Zhaopeng Qiu, Hao Wang, Hongchao Gu, Tingjia Shen, Chuan Qin, Chen Zhu, Hengshu Zhu, Qi Liu, et al. 2023. A survey on large language models for recommendation. *arXiv preprint arXiv:2305.19860* (2023). https://doi.org/10.48550/arXiv.2305.19860

[53] Linting Xue, Noah Constant, Adam Roberts, Mihir Kale, Rami Al-Rfou, Aditya Siddhant, Aditya Barua, and Colin Raffel. 2021. mT5: A Massively Multilingual Pre-trained Text-to-Text Transformer. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou (Eds.). Association for Computational Linguistics, Online, 483–498. https://doi.org/10.18653/v1/2021.naacl-main.41

[54] An Yan, Shuo Cheng, Wang-Cheng Kang, Mengting Wan, and Julian McAuley. 2019. CosRec: 2D Convolutional Neural Networks for Sequential Recommendation. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*. ACM, 2173–2176. https://doi.org/10.1145/3357384.3358113

[55] Jiahui Yu, Xin Li, Jing Yu Koh, Han Zhang, Ruoming Pang, James Qin, Alexander Ku, Yuanzhong Xu, Jason Baldridge, and Yonghui Wu. 2021. Vector-quantized image modeling with improved vqgan. *arXiv preprint arXiv:2110.04627* (2021). https://doi.org/10.48550/arXiv.2110.04627

[56] Junjie Zhang, Ruobing Xie, Yupeng Hou, Wayne Xin Zhao, Leyu Lin, and Ji-Rong Wen. 2023. Recommendation as instruction following: A large language model empowered recommendation approach. *arXiv preprint arXiv:2305.07001* (2023). https://doi.org/10.48550/arXiv.2305.07001

[57] Tingting Zhang, Pengpeng Zhao, Yanchi Liu, Victor S. Sheng, Jiajie Xu, Deqing Wang, Guanfeng Liu, and Xiaofang Zhou. 2019. Feature-level Deeper Self-Attention Network for Sequential Recommendation. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI-19*. International Joint Conferences on Artificial Intelligence Organization, 4320–4326. https://doi.org/10.24963/ijcai.2019/600

[58] Wayne Xin Zhao, Shanlei Mu, Yupeng Hou, Zihan Lin, Yushuo Chen, Xingyu Pan, Kaiyuan Li, Yujie Lu, Hui Wang, Changxin Tian, et al. 2021. Recbole: Towards a unified, comprehensive and efficient framework for recommendation algorithms. In *proceedings of the 30th acm international conference on information & knowledge management*. 4653–4664. https://doi.org/10.1145/3459637.3482016

[59] Kun Zhou, Hui Wang, Wayne Xin Zhao, Yutao Zhu, Sirui Wang, Fuzheng Zhang, Zhongyuan Wang, and Ji-Rong Wen. 2020. S3-Rec: Self-Supervised Learning for Sequential Recommendation with Mutual Information Maximization. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management* (Virtual Event, Ireland) *(CIKM '20)*. Association for Computing Machinery, New York, NY, USA, 1893–1902. https://doi.org/10.1145/3340531.3411954

[60] Yaochen Zhu, Liang Wu, Qi Guo, Liangjie Hong, and Jundong Li. 2024. Collaborative Large Language Model for Recommender Systems. In *Proceedings of the ACM on Web Conference 2024* (Singapore, Singapore) *(WWW '24)*. Association for Computing Machinery, New York, NY, USA, 3162–3172. https://doi.org/10.1145/3589334.3645347